

Simulate Your Network with NS2

Learn about Network Simulator-2, its installation and the execution of simple TCL files of network architecture.



Network Simulator-2 (NS2) is a discrete event network simulator that plays an important role in network research and development. It is one of the most widely used open source network simulators. It has been enhanced over the years, leading to a complete package consisting of various modules for functions such as routing, the transport layer protocol and applications.

To observe network performance, an easy scripting language can be used, by which you can configure the network as per your architecture and observe the results to check the correctness of your configuration.

NS2 plays the role of both emulator and simulator. The latter contains three types of discrete event schedulers: list, heap and hash-based calendar. NS2 provides default implementations for network nodes, links between nodes, routing algorithms, some transport level protocols (especially UDP and TCP) and some traffic generators. It also contains some useful utilities like the TCL debugger, simulation scenario generator and simulation topology generator. The TCL debugger is used to debug TCL scripts, which becomes necessary if one is using large scripts to control a simulation.

It is also possible to use NS2 as an emulator, which is currently supported by only FreeBSD. The NS2 emulator can be used to connect the tool to a live network. The

NS2 emulator works on two modes, i.e., the protocol mode and the opaque mode. In the protocol mode, the emulator interprets received traffic, whereas in the opaque mode, the received data is not interpreted.

An insight into the architecture of NS2

NS2 is primarily designed on two languages: C++ and Object-oriented Tool Command Language (OTCL). C++ is used for defining the internals of NS2 while OTCL is used to control the simulation as well as to schedule discrete events. C++ and OTCL are linked together using TCLCL. After the linking of C++ member variables to OTCL object variables by using the bind member function, C++ variables can be modified through OTCL directly. The main drawbacks of this approach are that the user has to know C++ as well as OTCL, and the debugging of simulations becomes more difficult.

After simulation, NS2 outputs either text-based or animation-based simulation results. To interpret these results graphically and interactively, tools such as NAM and XGraph are used, which are explained in the next section. To analyse some particular behaviour of the network, extract a relevant subset of the text-based data and transform it into a more understandable presentation. The basic architecture is shown in Figure 1, pictorially.